

Show how to implement mergeable heaps using linked lists in each of the following cases. Try to make each operation as efficient as possible. Analyze the running time of each operation in terms of the size of the dynamic set(s) being operated on.

- a. Lists are sorted.
- b. Lists are unsorted.
- c. Lists are unsorted, and dynamic sets to be merged are disjoint.

10-3 Searching a sorted compact list

We can represent a singly linked list with two arrays, *key* and *next*. Given the index *i* of an element, its value is stored in *key*[*i*], and the index of its successor is given by *next*[*i*], where *next*[*i*] = NIL for the last element. We also need the index *head* of the first element in the list. An *n*-element list stored in this way is **compact** if it is stored only in positions 1 through *n* of the *key* and *next* arrays.

Let's assume that all keys are distinct and that the compact list is also sorted, that is, $key[i] < key[next[i]]$ for all $i = 1, 2, \dots, n$ such that $next[i] \neq \text{NIL}$. Under these assumptions, you will show that the randomized algorithm COMPACT-LIST-SEARCH searches the list for key *k* in $O(\sqrt{n})$ expected time.

COMPACT-LIST-SEARCH(*key*, *next*, *head*, *n*, *k*)

```

1  i = head
2  while i ≠ NIL and key[i] < k
3      j = RANDOM(1, n)
4      if key[i] < key[j] and key[j] ≤ k
5          i = j
6          if key[i] == k
7              return i
8      i = next[i]
9  if i == NIL or key[i] > k
10     return NIL
11 else return i
```

If you ignore lines 3–7 of the procedure, you can see that it's an ordinary algorithm for searching a sorted linked list, in which index *i* points to each position of the list in turn. The search terminates once the index *i* “falls off” the end of the list or once $key[i] \geq k$. In the latter case, if $key[i] = k$, the procedure has found a key with the value *k*. If, however, $key[i] > k$, then the search will never find a key with the value *k*, so that terminating the search was the correct action.

Lines 3–7 attempt to skip ahead to a randomly chosen position j . Such a skip helps if $\text{key}[j]$ is larger than $\text{key}[i]$ and no larger than k . In such a case, j marks a position in the list that i would reach during an ordinary list search. Because the list is compact, we know that any choice of j between 1 and n indexes some element in the list.

Instead of analyzing the performance of COMPACT-LIST-SEARCH directly, you will analyze a related algorithm, COMPACT-LIST-SEARCH', which executes two separate loops. This algorithm takes an additional parameter t , which specifies an upper bound on the number of iterations of the first loop.

COMPACT-LIST-SEARCH'($\text{key}, \text{next}, \text{head}, n, k, t$)

```

1   $i = \text{head}$ 
2  for  $q = 1$  to  $t$ 
3       $j = \text{RANDOM}(1, n)$ 
4      if  $\text{key}[i] < \text{key}[j]$  and  $\text{key}[j] \leq k$ 
5           $i = j$ 
6          if  $\text{key}[i] == k$ 
7              return  $i$ 
8  while  $i \neq \text{NIL}$  and  $\text{key}[i] < k$ 
9       $i = \text{next}[i]$ 
10 if  $i == \text{NIL}$  or  $\text{key}[i] > k$ 
11     return NIL
12 else return  $i$ 
```

To compare the execution of the two algorithms, assume that the sequence of calls of $\text{RANDOM}(1, n)$ yields the same sequence of integers for both algorithms.

- a. Argue that for any value of t , COMPACT-LIST-SEARCH($\text{key}, \text{next}, \text{head}, n, k$) and COMPACT-LIST-SEARCH'($\text{key}, \text{next}, \text{head}, n, k, t$) return the same result and that the number of iterations of the **while** loop of lines 2–8 in COMPACT-LIST-SEARCH is at most the total number of iterations of both the **for** and **while** loops in COMPACT-LIST-SEARCH'.

In the call COMPACT-LIST-SEARCH'($\text{key}, \text{next}, \text{head}, n, k, t$), let X_t be the random variable that describes the distance in the linked list (that is, through the chain of *next* pointers) from position i to the desired key k after t iterations of the **for** loop of lines 2–7 have occurred.

- b. Argue that COMPACT-LIST-SEARCH'($\text{key}, \text{next}, \text{head}, n, k, t$) has an expected running time of $O(t + E[X_t])$.
- c. Show that $E[X_t] = \sum_{r=1}^n (1 - r/n)^t$. (Hint: Use equation (C.28) on page 1193.)

- d.* Show that $\sum_{r=0}^{n-1} r^t \leq n^{t+1}/(t+1)$. (*Hint:* Use inequality (A.18) on page 1150.)
- e.* Prove that $E[X_t] \leq n/(t+1)$.
- f.* Show that $\text{COMPACT-LIST-SEARCH}'(\text{key}, \text{next}, \text{head}, n, k, t)$ has an expected running time of $O(t + n/t)$.
- g.* Conclude that $\text{COMPACT-LIST-SEARCH}$ runs in $O(\sqrt{n})$ expected time.
- h.* Why do we assume that all keys are distinct in $\text{COMPACT-LIST-SEARCH}$? Argue that random skips do not necessarily help asymptotically when the list contains repeated key values.

Chapter notes

Aho, Hopcroft, and Ullman [6] and Knuth [259] are excellent references for elementary data structures. Many other texts cover both basic data structures and their implementation in a particular programming language. Examples of these types of textbooks include Goodrich and Tamassia [196], Main [311], Shaffer [406], and Weiss [452, 453, 454]. The book by Gonnet and Baeza-Yates [193] provides experimental data on the performance of many data-structure operations.

The origin of stacks and queues as data structures in computer science is unclear, since corresponding notions already existed in mathematics and paper-based business practices before the introduction of digital computers. Knuth [259] cites A. M. Turing for the development of stacks for subroutine linkage in 1947.

Pointer-based data structures also seem to be a folk invention. According to Knuth, pointers were apparently used in early computers with drum memories. The A-1 language developed by G. M. Hopper in 1951 represented algebraic formulas as binary trees. Knuth credits the IPL-II language, developed in 1956 by A. Newell, J. C. Shaw, and H. A. Simon, for recognizing the importance and promoting the use of pointers. Their IPL-III language, developed in 1957, included explicit stack operations.

11 Hash Tables

Many applications require a dynamic set that supports only the dictionary operations INSERT, SEARCH, and DELETE. For example, a compiler that translates a programming language maintains a symbol table, in which the keys of elements are arbitrary character strings corresponding to identifiers in the language. A hash table is an effective data structure for implementing dictionaries. Although searching for an element in a hash table can take as long as searching for an element in a linked list— $\Theta(n)$ time in the worst case—in practice, hashing performs extremely well. Under reasonable assumptions, the average time to search for an element in a hash table is $O(1)$. Indeed, the built-in dictionaries of Python are implemented with hash tables.

A hash table generalizes the simpler notion of an ordinary array. Directly addressing into an ordinary array takes advantage of the $O(1)$ access time for any array element. Section 11.1 discusses direct addressing in more detail. To use direct addressing, you must be able to allocate an array that contains a position for every possible key.

When the number of keys actually stored is small relative to the total number of possible keys, hash tables become an effective alternative to directly addressing an array, since a hash table typically uses an array of size proportional to the number of keys actually stored. Instead of using the key as an array index directly, we *compute* the array index from the key. Section 11.2 presents the main ideas, focusing on “chaining” as a way to handle “collisions,” in which more than one key maps to the same array index. Section 11.3 describes how to compute array indices from keys using hash functions. We present and analyze several variations on the basic theme. Section 11.4 looks at “open addressing,” which is another way to deal with collisions. The bottom line is that hashing is an extremely effective and practical technique: the basic dictionary operations require only $O(1)$ time on the average. Section 11.5 discusses the hierarchical memory systems of modern computer systems have and illustrates how to design hash tables that work well in such systems.

11.1 Direct-address tables

Direct addressing is a simple technique that works well when the universe U of keys is reasonably small. Suppose that an application needs a dynamic set in which each element has a distinct key drawn from the universe $U = \{0, 1, \dots, m-1\}$, where m is not too large.

To represent the dynamic set, you can use an array, or *direct-address table*, denoted by $T[0 : m-1]$, in which each position, or *slot*, corresponds to a key in the universe U . Figure 11.1 illustrates this approach. Slot k points to an element in the set with key k . If the set contains no element with key k , then $T[k] = \text{NIL}$.

The dictionary operations DIRECT-ADDRESS-SEARCH, DIRECT-ADDRESS-INSERT, and DIRECT-ADDRESS-DELETE on the following page are trivial to implement. Each takes only $O(1)$ time.

For some applications, the direct-address table itself can hold the elements in the dynamic set. That is, rather than storing an element's key and satellite data in an object external to the direct-address table, with a pointer from a slot in the table to the object, save space by storing the object directly in the slot. To indicate an empty slot, use a special key. Then again, why store the key of the object at all? The index of the object *is* its key! Of course, then you'd need some way to tell whether slots are empty.

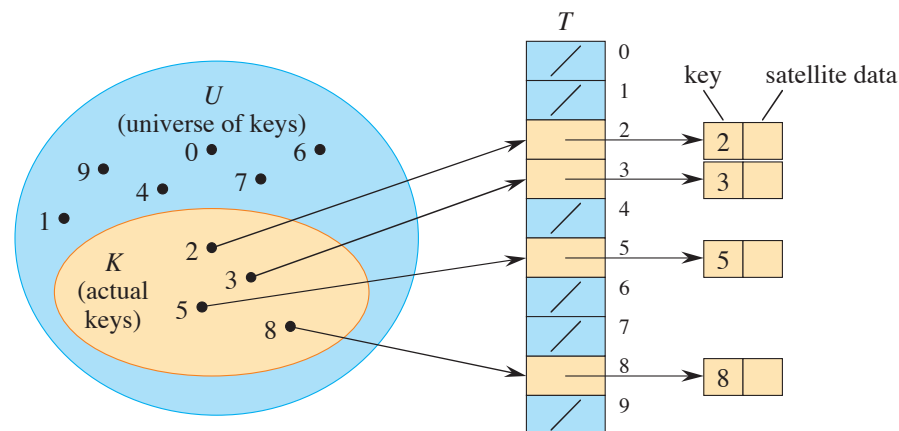


Figure 11.1 How to implement a dynamic set by a direct-address table T . Each key in the universe $U = \{0, 1, \dots, 9\}$ corresponds to an index into the table. The set $K = \{2, 3, 5, 8\}$ of actual keys determines the slots in the table that contain pointers to elements. The other slots, in blue, contain NIL.

```
DIRECT-ADDRESS-SEARCH( $T, k$ )
```

```
1  return  $T[k]$ 
```

```
DIRECT-ADDRESS-INSERT( $T, x$ )
```

```
1   $T[x.key] = x$ 
```

```
DIRECT-ADDRESS-DELETE( $T, x$ )
```

```
1   $T[x.key] = \text{NIL}$ 
```

Exercises

11.1-1

A dynamic set S is represented by a direct-address table T of length m . Describe a procedure that finds the maximum element of S . What is the worst-case performance of your procedure?

11.1-2

A *bit vector* is simply an array of bits (each either 0 or 1). A bit vector of length m takes much less space than an array of m pointers. Describe how to use a bit vector to represent a dynamic set of distinct elements drawn from the set $\{0, 1, \dots, m-1\}$ and with no satellite data. Dictionary operations should run in $O(1)$ time.

11.1-3

Suggest how to implement a direct-address table in which the keys of stored elements do not need to be distinct and the elements can have satellite data. All three dictionary operations (INSERT, DELETE, and SEARCH) should run in $O(1)$ time. (Don't forget that DELETE takes as an argument a pointer to an object to be deleted, not a key.)

★ 11.1-4

Suppose that you want to implement a dictionary by using direct addressing on a *huge* array. That is, if the array size is m and the dictionary contains at most n elements at any one time, then $m \gg n$. At the start, the array entries may contain garbage, and initializing the entire array is impractical because of its size. Describe a scheme for implementing a direct-address dictionary on a huge array. Each stored object should use $O(1)$ space; the operations SEARCH, INSERT, and DELETE should take $O(1)$ time each; and initializing the data structure should take $O(1)$ time. (*Hint:* Use an additional array, treated somewhat like a stack whose size is the number of keys actually stored in the dictionary, to help determine whether a given entry in the huge array is valid or not.)

11.2 Hash tables

The downside of direct addressing is apparent: if the universe U is large or infinite, storing a table T of size $|U|$ may be impractical, or even impossible, given the memory available on a typical computer. Furthermore, the set K of keys *actually stored* may be so small relative to U that most of the space allocated for T would be wasted.

When the set K of keys stored in a dictionary is much smaller than the universe U of all possible keys, a hash table requires much less storage than a direct-address table. Specifically, the storage requirement reduces to $\Theta(|K|)$ while maintaining the benefit that searching for an element in the hash table still requires only $O(1)$ time. The catch is that this bound is for the *average-case time*,¹ whereas for direct addressing it holds for the *worst-case time*.

With direct addressing, an element with key k is stored in slot k , but with hashing, we use a **hash function** h to compute the slot number from the key k , so that the element goes into slot $h(k)$. The hash function h maps the universe U of keys into the slots of a **hash table** $T[0 : m - 1]$:

$$h : U \rightarrow \{0, 1, \dots, m - 1\} ,$$

where the size m of the hash table is typically much less than $|U|$. We say that an element with key k **hashes** to slot $h(k)$, and we also say that $h(k)$ is the **hash value** of key k . Figure 11.2 illustrates the basic idea. The hash function reduces the range of array indices and hence the size of the array. Instead of a size of $|U|$, the array can have size m . An example of a simple, but not particularly good, hash function is $h(k) = k \bmod m$.

There is one hitch, namely that two keys may hash to the same slot. We call this situation a **collision**. Fortunately, there are effective techniques for resolving the conflict created by collisions.

Of course, the ideal solution is to avoid collisions altogether. We might try to achieve this goal by choosing a suitable hash function h . One idea is to make h appear to be “random,” thus avoiding collisions or at least minimizing their number. The very term “to hash,” evoking images of random mixing and chopping, captures the spirit of this approach. (Of course, a hash function h must be deterministic in that a given input k must always produce the same output $h(k)$.) Because $|U| > m$, however, there must be at least two keys that have the same hash value,

¹ The definition of “average-case” requires care—are we assuming an input distribution over the keys, or are we randomizing the choice of hash function itself? We’ll consider both approaches, but with an emphasis on the use of a randomly chosen hash function.

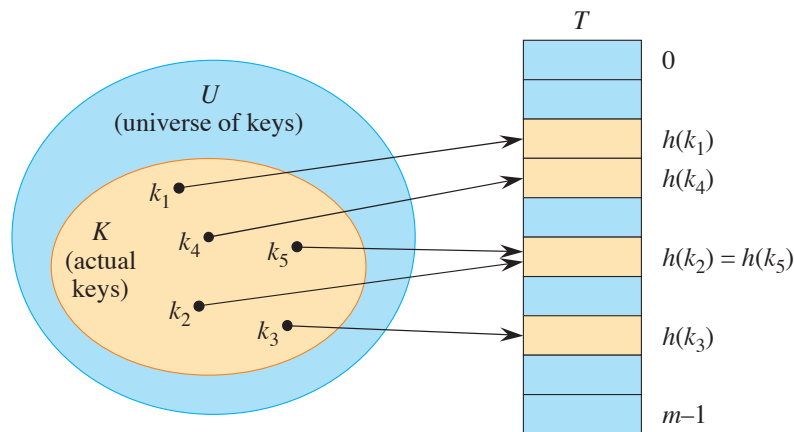


Figure 11.2 Using a hash function h to map keys to hash-table slots. Because keys k_2 and k_5 map to the same slot, they collide.

and avoiding collisions altogether is impossible. Thus, although a well-designed, “random”-looking hash function can reduce the number of collisions, we still need a method for resolving the collisions that do occur.

The remainder of this section first presents a definition of “independent uniform hashing,” which captures the simplest notion of what it means for a hash function to be “random.” It then presents and analyzes the simplest collision resolution technique, called chaining. Section 11.4 introduces an alternative method for resolving collisions, called open addressing.

Independent uniform hashing

An “ideal” hashing function h would have, for each possible input k in the domain U , an output $h(k)$ that is an element randomly and independently chosen uniformly from the range $\{0, 1, \dots, m-1\}$. Once a value $h(k)$ is randomly chosen, each subsequent call to h with the same input k yields the same output $h(k)$.

We call such an ideal hash function an *independent uniform hash function*. Such a function is also often called a *random oracle* [43]. When hash tables are implemented with an independent uniform hash function, we say we are using *independent uniform hashing*.

Independent uniform hashing is an ideal theoretical abstraction, but it is not something that can reasonably be implemented in practice. Nonetheless, we’ll analyze the efficiency of hashing under the assumption of independent uniform hashing and then present ways of achieving useful practical approximations to this ideal.

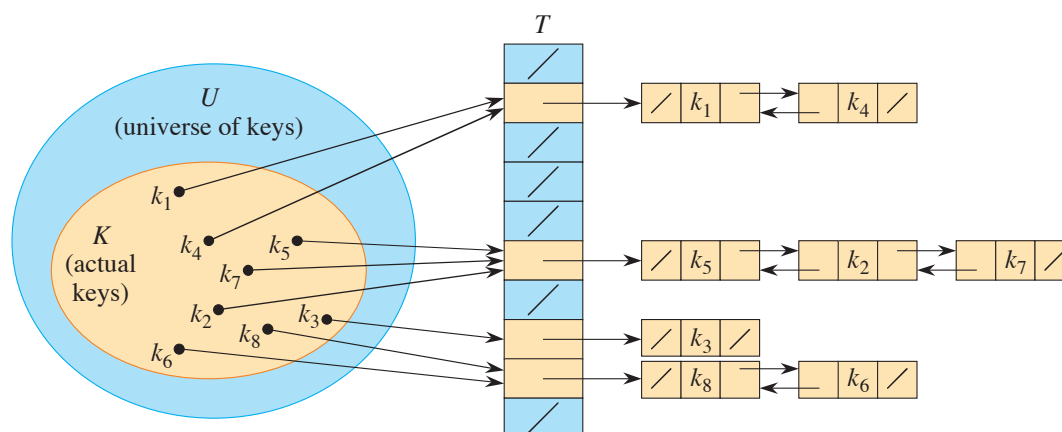


Figure 11.3 Collision resolution by chaining. Each nonempty hash-table slot $T[j]$ points to a linked list of all the keys whose hash value is j . For example, $h(k_1) = h(k_4)$ and $h(k_5) = h(k_2) = h(k_7)$. The list can be either singly or doubly linked. We show it as doubly linked because deletion may be faster that way when the deletion procedure knows which list element (not just which key) is to be deleted.

Collision resolution by chaining

At a high level, you can think of hashing with chaining as a nonrecursive form of divide-and-conquer: the input set of n elements is divided randomly into m subsets, each of approximate size n/m . A hash function determines which subset an element belongs to. Each subset is managed independently as a list.

Figure 11.3 shows the idea behind **chaining**: each nonempty slot points to a linked list, and all the elements that hash to the same slot go into that slot's linked list. Slot j contains a pointer to the head of the list of all stored elements with hash value j . If there are no such elements, then slot j contains NIL.

When collisions are resolved by chaining, the dictionary operations are straightforward to implement. They appear on the next page and use the linked-list procedures from Section 10.2. The worst-case running time for insertion is $O(1)$. The insertion procedure is fast in part because it assumes that the element x being inserted is not already present in the table. To enforce this assumption, you can search (at additional cost) for an element whose key is $x.key$ before inserting. For searching, the worst-case running time is proportional to the length of the list. (We'll analyze this operation more closely below.) Deletion takes $O(1)$ time if the lists are doubly linked, as in Figure 11.3. (Since CHAINED-HASH-DELETE takes as input an element x and not its key k , no search is needed. If the hash table supports deletion, then its linked lists should be doubly linked in order to delete an item quickly. If the lists were only singly linked, then by Exercise 10.2-1, deletion

```

CHAINED-HASH-INSERT( $T, x$ )
1  LIST-PREPEND( $T[h(x.key)], x$ )

CHAINED-HASH-SEARCH( $T, k$ )
1  return LIST-SEARCH( $T[h(k)], k$ )

CHAINED-HASH-DELETE( $T, x$ )
1  LIST-DELETE( $T[h(x.key)], x$ )

```

could take time proportional to the length of the list. With singly linked lists, both deletion and searching would have the same asymptotic running times.)

Analysis of hashing with chaining

How well does hashing with chaining perform? In particular, how long does it take to search for an element with a given key?

Given a hash table T with m slots that stores n elements, we define the *load factor* α for T as n/m , that is, the average number of elements stored in a chain. Our analysis will be in terms of α , which can be less than, equal to, or greater than 1.

The worst-case behavior of hashing with chaining is terrible: all n keys hash to the same slot, creating a list of length n . The worst-case time for searching is thus $\Theta(n)$ plus the time to compute the hash function—no better than using one linked list for all the elements. We clearly don't use hash tables for their worst-case performance.

The average-case performance of hashing depends on how well the hash function h distributes the set of keys to be stored among the m slots, on the average (meaning with respect to the distribution of keys to be hashed and with respect to the choice of hash function, if this choice is randomized). Section 11.3 discusses these issues, but for now we assume that any given element is equally likely to hash into any of the m slots. That is, the hash function is *uniform*. We further assume that where a given element hashes to is *independent* of where any other elements hash to. In other words, we assume that we are using *independent uniform hashing*.

Because hashes of distinct keys are assumed to be independent, independent uniform hashing is *universal*: the chance that any two distinct keys k_1 and k_2 collide is at most $1/m$. Universality is important in our analysis and also in the specification of universal families of hash functions, which we'll see in Section 11.3.2.

For $j = 0, 1, \dots, m - 1$, denote the length of the list $T[j]$ by n_j , so that